Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

Tutorial 7 : Exploiter la station dans la durée, modèle versionné et signé, mise à jour A/B, surveillance de dérive et commandes authentifiées sur la Portenta H7 avec OpenMV

Séries de tuto sur Portenta H7, lecture 7 « Industrialiser et défendre l'IA embarquée »

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

2 novembre 2026

Objectifs du Tutorial

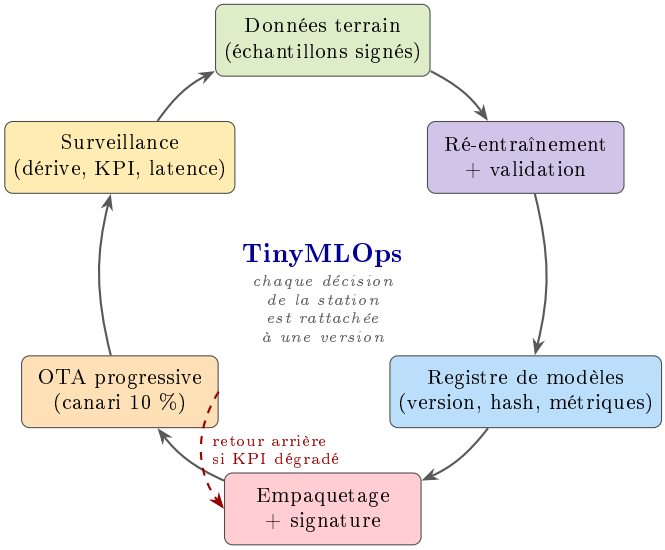
Au Tutorial 6, la station triait. Ici, vous lui donnez les quatre propriétés qui séparent un démonstrateur d'un produit : elle **sait quel modèle elle exécute** et refuse un modèle incohérent, elle **se met à jour sans risque** et revient en arrière si la nouvelle version échoue, elle **détecte que le monde a changé** sans étiquettes, et elle **n'obéit qu'aux commandes authentifiées**. Enfin, vous calculez l'OEE à partir de ses propres messages.
À la fin de ce tutoriel, vous serez capable de :

- Produire sur PC un manifeste de modèle (version, SHA-256, paramètres d'entrée) et le signer.
- Charger un modèle sur la Portenta après vérification complète du manifeste.
- Implémenter une mise à jour A/B du modèle avec auto-test et retour arrière automatique.
- Surveiller la dérive par PSI (luminosité) et Page-Hinkley (confiance) directement sur la carte.
- Authentifier les commandes BLE par HMAC-SHA256 avec compteur anti-rejeu.
- Calculer D, P, Q et l'OEE à partir du journal JSON de la station.

Durée indicative : 1 h 45 (à faire **avant** la séance 7 : il sert de base à l'épreuve pratique). Pré-requis : Tutorials 1, 5 et 6, fichiers `trash_logits_int8.tflite`, `labels.txt`, `decision.py`, `profil.py`.

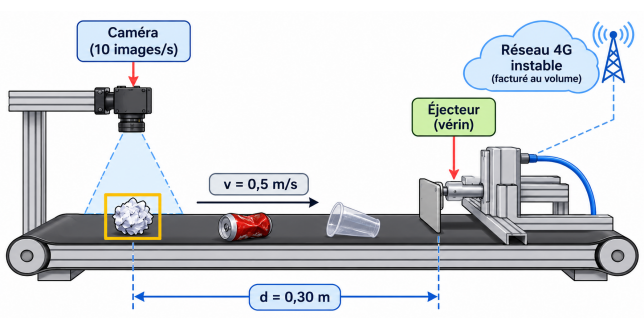
1 Introduction : du démonstrateur au produit

Un modèle déployé n'est pas figé : il sera remplacé, son environnement va changer et quelqu'un essaiera, par erreur ou malveillance, de lui faire faire autre chose. Le cycle ci-dessous résume ce qu'il faut outiller.



La boucle TinyMLOps. Dans ce tutoriel, vous réalisez sur la Portenta les maillons « signature », « OTA » et « surveillance ».

Ce que nous allons traiter

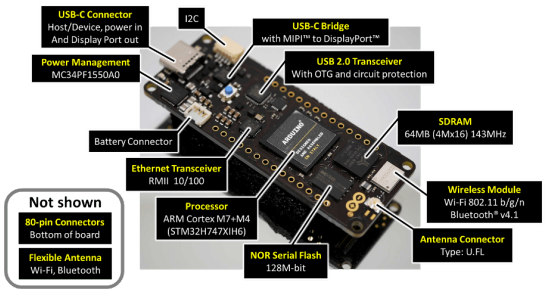


La station EcoTri 4.0 telle qu'à l'issue du jalon J6. Le matériel ne change pas : tout le travail porte sur le logiciel d'exploitation.

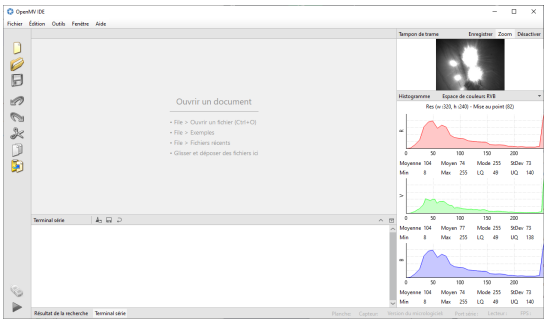
2 Pré-requis

Matériel

- Carte **Arduino Portenta H7** et **Vision Shield**, câble USB-C.
- Une lampe d'appoint (pour provoquer une dérive), quelques objets du fil rouge.
- Un smartphone avec nRF Connect (étape 6).



Portenta H7 : l'élément sécurisé SE050 se trouve sur la carte



OpenMV IDE : le disque de la carte reçoit les fichiers du modèle

Logiciel

1. **OpenMV IDE 4.x**, firmware **4.5 ou plus** (modules `ml`, `hashlib`, `bluetooth`).
2. **Python 3** sur le PC (bibliothèques standard `hashlib`, `hmac`, `json`) ; Colab convient aussi.
3. Vos fichiers du Tutorial 6 ; deux versions du modèle (par exemple la version INT8 du Tutorial 5 et la version calibrée du Tutorial 6), que nous appellerons v4.0 et v4.1.

3 Étapes détaillées du tutoriel

Étape 1 : Générer le manifeste sur le PC

Le manifeste décrit tout ce qui doit concorder entre le modèle et la station. Sur le PC, dans le dossier contenant le modèle :

```
# pc_manifeste.py -- crée manifeste_<v>.json et sa signature manifeste_<v>.sig
import hashlib, hmac, json, sys, tensorflow as tf

CLE_SIGNATURE = open("cle_signature.bin", "rb").read()   # 32 octets, JAMAIS dans le
    dépôt Git

def sha256(chemin):
    h = hashlib.sha256()
    with open(chemin, "rb") as f:
        for bloc in iter(lambda: f.read(4096), b""): h.update(bloc)
    return h.hexdigest()

modele, version = sys.argv[1], sys.argv[2]                # ex. trash_logits_int8.
    tflite 4.1.0
it = tf.lite.Interpreter(model_path=modele); ie = it.get_input_details()[0]
s, z = ie["quantization"]
man = {"modele": "mnv2a035-int8", "version": version, "fichier": modele,
       "sha256": sha256(modele), "taille": len(open(modele, "rb").read()),
       "entree": {"forme": [int(v) for v in ie["shape"]], "s": float(s), "z": int(z)
    },
       "classes": [l.strip() for l in open("labels.txt")],
       "T_cal": 1.8, "theta": 0.70, "firmware_min": "4.5.0"}
corps = json.dumps(man, indent=1).encode()                # octets exacts signés
open("manifeste_" + version + ".json", "wb").write(corps)
open("manifeste_" + version + ".sig", "w").write(hmac.new(CLE_SIGNATURE, corps,
    hashlib.sha256).hexdigest())
print("OK", man["sha256"][:16], "...")
```

1. Créez une clé : `python -c "import os; open('cle_signature.bin','wb').write(os.urandom(32))"`.

2. Générez les manifestes de v4.0 et v4.1, puis renommez-les en `manifeste_A.*` et `manifeste_B.*`. Ouvrez-les : retrouvez-vous s et z du Tutorial 5 ?

Astuce

Nous signons ici avec un HMAC (clé symétrique), disponible partout en MicroPython. En production, on signe avec une clé **privée** ECDSA P-256 qui ne quitte jamais le serveur, et la station ne détient que la clé **publique** (vérification par l'élément sécurisé SE050 de la Portenta, ou par Secure Boot sur l'ESP32-S3). Avec un HMAC, quiconque extrait la clé de la carte peut signer : c'est la limite de cette version pédagogique.

Étape 2 : Charger un modèle seulement s'il est cohérent

Copiez sur le disque de la carte `modele_A.tflite` (v4.0), son manifeste `manifeste_A.json` et sa signature `manifeste_A.sig`, `labels.txt` et un fichier `cle.bin` contenant la même clé. Créez `chargeur.py` :

```
# chargeur.py -- vérification complète avant ml.Model()
import hashlib, json, ubinascii, ml, os
CLE = open("cle.bin", "rb").read()

def hmac_sha256(k, m):                                  # RFC 2104
    k = k + b"\x00" * (64 - len(k))
    i = hashlib.sha256(bytes(b ^ 0x36 for b in k) + m).digest()
    return hashlib.sha256(bytes(b ^ 0x5C for b in k) + i).digest()

def egal_temps_constant(a, b):
    if len(a) != len(b): return False
    r = 0
    for x, y in zip(a, b): r |= x ^ y
    return r == 0

def sha256_fichier(chemin):
    h = hashlib.sha256()
    with open(chemin, "rb") as f:
        while True:
            b = f.read(4096)
            if not b: break
            h.update(b)
    return ubinascii.hexlify(h.digest()).decode()

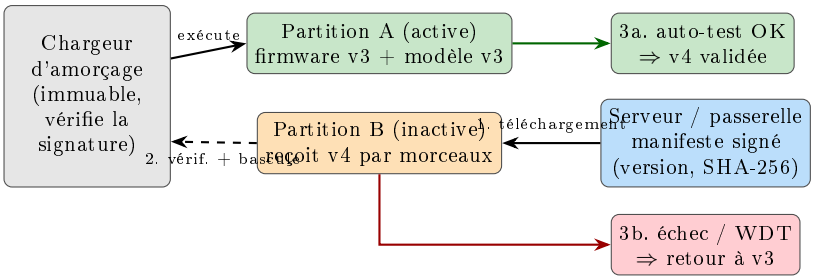
def charger(fichier_modele, fichier_manifeste):
    brut = open(fichier_manifeste, "rb").read()             # on vérifie les octets,
    pas le JSON relu
    sig = ubinascii.unhexlify(open(fichier_manifeste[:-5] + ".sig").read().strip())
    if not egal_temps_constant(hmac_sha256(CLE, brut), sig):
        raise ValueError("signature invalide")
    man = json.loads(brut)
    if sha256_fichier(fichier_modele) != man["sha256"]:
        raise ValueError("empreinte du modèle incorrecte")
    labels = [l.strip() for l in open("labels.txt")]
    if labels != man["classes"]:
        raise ValueError("labels.txt ne correspond pas au manifeste")
    net = ml.Model(fichier_modele, load_to_fb=True)
    if list(net.input_shape[0]) != man["entree"]["forme"]:
        raise ValueError("forme d'entrée inattendue %s" % str(net.input_shape))
    return net, man, labels
```

1. Testez `net, man, labels = charger("modele_A.tflite", "manifeste_A.json")` : la version s'affiche-t-elle ?
2. **Attaquez votre propre station** : (a) modifiez un octet du modèle avec un éditeur hexadécimal ; (b) changez `"theta": 0.7` en `0.3` dans le manifeste ; (c) inversez deux lignes de `labels.txt`. Quelle vérification bloque chaque cas ?
3. Mesurez le temps de `sha256_fichier` pour un modèle de ≈ 400 Ko. Est-il acceptable au démarrage ?

Question

Pourquoi l'inversion de deux lignes de `labels.txt` est-elle plus dangereuse qu'un modèle corrompu ? (Indice : lequel des deux produit une erreur visible ?)

Étape 3 : Mise à jour A/B avec auto-test et retour arrière



Le même schéma appliqué aux fichiers du modèle : deux emplacements A et B, un pointeur `actif.json`, une confirmation après auto-test.

Le pointeur `actif.json` contient `{"actif": "A", "essai": null, "tentatives": 0}`. Une mise à jour écrit le nouveau modèle dans l'emplacement inactif, puis positionne `essai`. Au démarrage :

```
# ota.py -- sélection du modèle au démarrage, auto-test, confirmation ou retour arrière
import json, sensor, os
from chargeur import charger
from decision import softmax_t

def lire():  return json.load(open("actif.json"))
def ecrire(e):
    json.dump(e, open("actif.tmp", "w")); os.rename("actif.tmp", "actif.json")   # écriture atomique

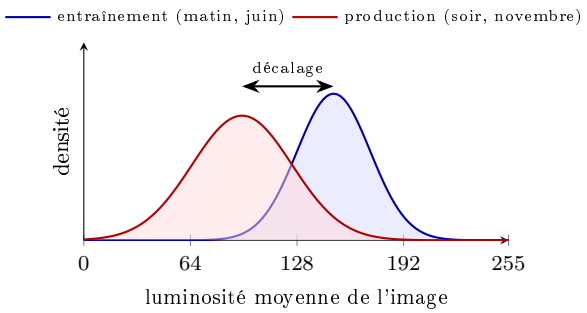
def autotest(net, man, labels):
    """Images de référence étiquetées stockées sur la carte : ref/metal_1.bmp ..."""
    import image
    ok = n = 0
    for f in os.listdir("ref"):
        attendu = f.split("_")[0]
        p = softmax_t(net.predict([image.Image("ref/" + f)])[0].flatten().tolist(), man["T_cal"])
        ok += labels[p.index(max(p))] == attendu; n += 1
    return n > 0 and ok / n >= 0.9

def demarrer():
    e = lire()
```

```
    if e["essai"]:                                  # une nouvelle version attend
        e["tentatives"] += 1; ecrire(e)
        try:
            if e["tentatives"] > 2: raise ValueError("trop de tentatives")
            s = e["essai"]
            net, man, labels = charger("modele_%s.tflite" % s, "manifeste_%s.json" %
s)
            if not autotest(net, man, labels): raise ValueError("auto-test échoué")
            e.update(actif=s, essai=None, tentatives=0); ecrire(e)
            print("Version", man["version"], "confirmée"); return net, man, labels
        except Exception as err:
            print("Retour arrière :", err)
            e.update(essai=None, tentatives=0); ecrire(e)
    a = e["actif"]
    return charger("modele_%s.tflite" % a, "manifeste_%s.json" % a)
```

1. Préparez le dossier `ref/` avec 8 images (2 par classe) enregistrées en BMP depuis la caméra (Tutorial 6, étape 1), nommées `classe_n.bmp`.
2. Déposez v4.1 en `modele_B.tflite` + `manifeste_B.json` + `manifeste_B.sig`, positionnez `"essai": "B"` et redémarrez : la version 4.1 est-elle confirmée ?
3. Recommencez avec un modèle B volontairement mauvais (par exemple le modèle avec les labels mélangés, ou un modèle entraîné une seule époque) : vérifiez le retour arrière automatique vers A.
4. Pourquoi le compteur `tentatives` est-il incrémenté **avant** le chargement ? Que se passerait-il si le nouveau modèle bloquait la carte (épuisement de la mémoire) sans lever d'exception ? Ajoutez un `machine.WDT(timeout=8000)` pour couvrir ce cas.

Étape 4 : Surveiller la dérive sur la carte



Vous allez provoquer volontairement ce décalage avec la lampe et vérifier qu'il est détecté.

a) Référence. Dans les conditions normales, laissez tourner la station sur une centaine d'objets et enregistrez l'histogramme de luminosité en 8 classes, ainsi que la moyenne et l'écart-type de $p_{\max}$:

```
# derive.py -- PSI (entrée) et Page-Hinkley (sortie), O(1) mémoire par objet
import math, json
B = 8
def classe_lum(img): return min(B - 1, int(img.get_statistics().mean() * B / 256))

class PSI:
    def __init__(self, ref, fenetre=200):
        self.ref, self.n, self.h, self.fenetre = ref, 0, [0] * B, fenetre
    def ajouter(self, b):
```

```
        self.h[b] += 1; self.n += 1
        if self.n < self.fenetre: return None
        q = [max(v / self.n, 1e-4) for v in self.h]
        psi = sum((qi - pi) * math.log(qi / pi) for qi, pi in zip(q, self.ref))
        self.n, self.h = 0, [0] * B
        return psi

class PageHinkley:          # détecte une BAISSE de la confiance moyenne
    def __init__(self, delta=0.005, lam=0.5):
        self.d, self.l = delta, lam; self.n = 0; self.moy = 0.0; self.m = 0.0; self.M
     = 0.0
    def ajouter(self, x):
        x = -x                                                  # baisse de x = hausse
    de -x
        self.n += 1; self.moy += (x - self.moy) / self.n
        self.m += x - self.moy - self.d; self.M = min(self.M, self.m)
        return (self.m - self.M) > self.l

def calibrer(histo):        # histo : comptes de la phase de référence
    n = sum(histo); ref = [max(v / n, 1e-4) for v in histo]
    json.dump({"ref": ref}, open("derive_ref.json", "w")); return ref
```

b) Intégration dans la boucle du Tutorial 6, après chaque décision :

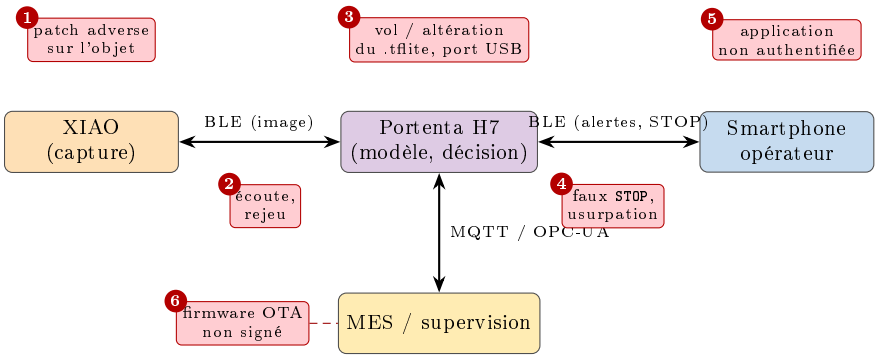
```
psi = PSI(json.load(open("derive_ref.json"))["ref"]); ph = PageHinkley()
# ... dans l'état NOTIFIER :
v = psi.ajouter(classe_lum(img))
if v is not None:
    msg["derive"] = {"psi": round(v, 3)}
    if v > 0.25: LED(1).on()                                    # alerte dérive d'entrée
if ph.ajouter(p[k]):
    msg["derive"] = dict(msg.get("derive", {}), ph=True)        # alerte confiance
    ph = PageHinkley()                                          # ré-armement
```

1. Avec une fenêtre de 50 objets (pour aller vite), relevez le PSI dans les conditions de référence. Doit-il être proche de 0 ? Pourquoi n'est-il pas exactement nul ?
2. Allumez la lampe (ou masquez une partie de l'éclairage) : au bout de combien d'objets le PSI dépasse-t-il 0,25 ? Page-Hinkley se déclenche-t-il, et après combien d'objets ?
3. Faites varier λ (0,2 ; 0,5 ; 1,0) : relevez le délai de détection et le nombre de fausses alertes sur 200 objets sans perturbation. Tracez le compromis.

Question

Un PSI élevé sans baisse de $p_{\max}$, et inversement une baisse de $p_{\max}$ avec un PSI faible : que suggère chacun des deux cas (covariable, concept, objet nouveau) ?

Étape 5 : Commandes authentifiées et anti-rejeu



Vous traitez ici la menace n° 4 (fausse commande `STOP`) au niveau applicatif.

Ajoutez au module BLE du TP3 (caractéristique d'écriture des commandes) la vérification suivante :

```
# commandes.py -- format "CMD|compteur|tag_hex16"
import ubinascii, json
from chargeur import hmac_sha256, egal_temps_constant, CLE
try:    dernier = json.load(open("cpt.json"))["cpt"]            # survit aux redé
    marrages
except: dernier = 0

def verifier(trame):
    global dernier
    try:
        cmd, cpt, tag = trame.decode().split("|"); cpt = int(cpt)
        attendu = hmac_sha256(CLE, ("%s|%d" % (cmd, cpt)).encode())[:16]
        if not egal_temps_constant(ubinascii.unhexlify(tag), attendu): return None, "
    tag"
        if cpt <= dernier: return None, "rejeu"
        dernier = cpt; json.dump({"cpt": cpt}, open("cpt.json", "w"))
        return cmd, "ok"
    except Exception:
        return None, "format"
```

Côté PC, générez des commandes valides (à coller dans nRF Connect, en texte) :

```
import hmac, hashlib, sys
k = open("cle_signature.bin", "rb").read(); cmd, cpt = sys.argv[1], int(sys.argv[2])
print("%s|%d|%s" % (cmd, cpt, hmac.new(k, f"{cmd}|{cpt}".encode(), hashlib.sha256).
    hexdigest()[:32]))
```

1. Envoyez `STOP|1|...` valide : le tapis s'arrête. Renvoyez exactement la même trame : quelle réponse ?
2. Envoyez `STOP` seul, puis `STOP|2|0000...` : le refus est-il journalisé avec sa raison ?
3. Que se passe-t-il si `cpt.json` est effacé ? Proposez une parade (indice : compteur en mémoire non volatile protégée, ou horodatage signé avec fenêtre).

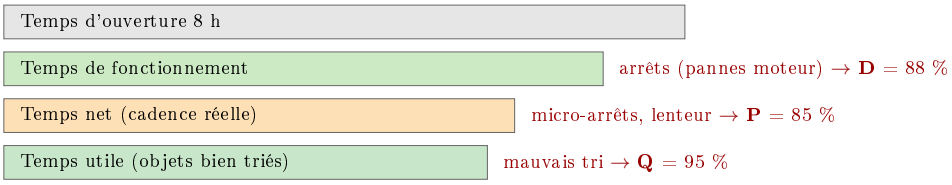
Astuce

Ne renvoyez pas au téléphone la raison exacte du refus (« tag » ou « rejeu ») : un attaquant s'en servirait pour affiner ses essais. Journalisez-la localement et répondez seulement `REFUS`.

Étape 6 : Calculer l'OEE à partir du journal

Exportez le journal des notifications de nRF Connect (ou les `print` de l'IDE) dans `journal.jsonl`, une ligne JSON par message. Sur le PC :

```
# pc_oee.py -- D, P, Q, OEE à partir des messages de la station
import json
msgs = [json.loads(l) for l in open("journal.jsonl") if l.strip().startswith("{")]
T_ouv = (msgs[-1]["t"] - msgs[0]["t"]) / 1000                    # s (champ "t" en ms)
arrets = 0; debut = None
for m in msgs:                                                    # durée cumulée des é
    tats ARRET
    if m["etat"] == "ARRET" and debut is None: debut = m["t"]
    if m["etat"] != "ARRET" and debut is not None: arrets += (m["t"] - debut) / 1000;
    debut = None
T_fonct = T_ouv - arrets
objets = [m for m in msgs if m["etat"] in ("TRIER", "INCONNU")]
N, t_cycle = len(objets), 1.0                                     # cadence nominale : 1
    objet/s
bons = sum(1 for m in objets if m["etat"] == "TRIER" and m.get("verite", m["classe"])
    == m["classe"])
D, P, Q = T_fonct / T_ouv, N * t_cycle / T_fonct, bons / N
print(f"D={D:.3f}  P={P:.3f}  Q={Q:.3f}  OEE={D*P*Q:.3f}")
```



OEE = D × P × Q = 0,88 × 0,85 × 0,95 ≈ 71 %

Représentez vos trois facteurs sous cette forme dans votre support de soutenance.

1. Faites tourner la station 10 minutes avec un arrêt simulé (son « moteur anormal ») d'une minute. Calculez D, P, Q.
2. Ici, les objets envoyés au bac « inconnu » comptent comme non conformes dans Q. Est-ce justifié ? Discutez avec le coût de rejet du Tutorial 6.
3. Le champ `verite` n'existe pas en production. Comment estimeriez-vous Q sur le terrain ? (Indice : échantillonnage et contrôle manuel.)

4 Pour aller plus loin

— Harvard *Machine Learning Systems*, chapitres « ML Operations » et « Security & Privacy » : mlsysbook.ai.

— MCUboot, chargeur d'amorçage sécurisé et mises à jour A/B : docs.mcuboot.com.

— Espressif, Secure Boot v2, Flash Encryption et OTA pour l'ESP32-S3 : docs.espressif.com.

— Arduino, utilisation de l'élément sécurisé SE050 de la Portenta H7 : docs.arduino.cc.

— Documentation OpenMV (`ml`, `hashlib`, `bluetooth`, `machine.WDT`) : docs.openmv.io.